Information Retrieval and Extraction — Assignment 2: Learning from Click-Logs on EB-NeRD and MIND

Piyush Kumar Agarwal(2025201082)
KSPSVLN Siddhardha Kumar Kavuri(2025201061)

Code: `https://github.com/piyukr2/IRE_assignment_1/tree/assignment-2`

1. What we built

A two-stage retrieve-then-rank pipeline on both datasets. Stage one is Assignment 1's candidate generator (BM25 and multilingual sentence embeddings). Stage two is a re-ranker trained on click-log features. Everything is split *temporally*: the re-ranker is fitted on the train period, stopped early on the validation period, and reported on the test period (2,000 test impressions per dataset, the same set for every model). We evaluate on `mind_small` and `ebnerd_small`.

What is re-ranked. Each impression's own candidate list (the serving path in Section 5 uses an ANN top-200, $K = 200$).

Features (nine, shared by both hand-crafted re-rankers):

- *Retrieval:* `bm25_score`, `semantic_score`.
- *Click history:* `n_clicks_in_history`, and `recency_weighted_click_count` (exponential decay, closed form $(1 - d^n)/(1 - d)$).
- *Article:* `popularity_train` (clicks in the training split only), `category_match` with the user's history, `has_published_time`.
- *Session (EB-NeRD only):* `session_click_count`, `position_in_session`. MIND has no sessions, so these are absent there rather than replaced by a proxy.

Freshness is not a model input (MIND ships no publication time). Dwell time is post-click, so it appears only in the ablation of Section 7. Behaviour window: a user's history for an impression contains only clicks strictly before its timestamp.

Re-rankers. (A) LightGBM `lambdarank`; (B) a small MLP, pointwise, on the same nine features; (C) an NRMS-style neural ranker over the click history (Section 3). All trainers are seeded and deterministic.

2. Re-ranker results: before and after (Q2)

Dataset	Model	AUC	MRR	nDCG@5	nDCG@10	Δ nDCG@10 [95% CI]
`ebnerd_small`	Retrieval only	0.5505	0.3475	0.3867	0.4676	
	GBDT (LightGBM)	0.4716	0.2770	0.3098	0.4045	−0.0632 [−0.0745, −0.0511]
	MLP	0.5370	0.3285	0.3704	0.4536	−0.0140 [−0.0230, −0.0050]
`mind_small`	Retrieval only	0.5985	0.3187	0.3039	0.3597	
	GBDT (LightGBM)	0.6125	0.3390	0.3184	0.3771	+0.0174 [+0.0062, +0.0286]
	MLP	0.6169	0.3446	0.3225	0.3800	+0.0203 [+0.0096, +0.0317]

Table 1: Test split, 2,000 impressions. Δ is the paired-bootstrap difference from "Retrieval only"; "n.s." marks an interval that includes zero; unmarked intervals exclude zero.

Observation. Both re-rankers beat retrieval-only on MIND (intervals exclude zero) and fall below it on EB-NeRD.

3. Baseline reproduced, then improved (Q3)

Reproduction. NRMS scores the dot product of a news vector and a user vector. We implement the click-level half as published (multi-head self-attention over past clicks, then additive attention) and replace the word-level half with the pinned 384-dimensional sentence embeddings from Assignment 1. Training is listwise (softmax over one clicked and four sampled non-clicked candidates). Hyperparameters are our own declared choices. The baseline is the retrieval-only `semantic_score`.

The one principled change: category-aware encoding. A learned category embedding is concatenated to every article vector; the two NRMS models differ only by it (a test asserts this).

Dataset	Model	AUC	MRR	nDCG@5	nDCG@10	Δ nDCG@10 [95% CI]
ebnerd_small	Retrieval only	0.5505	0.3475	0.3867	0.4676	
	NRMS reproduction	0.5631	0.3556	0.3968	0.4736	+0.0060 [−0.0070, +0.0205] n.s.
	NRMS + category	0.5895	0.3728	0.4193	0.4897	+0.0221 [+0.0081, +0.0362]
mind_small	Retrieval only	0.5985	0.3187	0.3039	0.3597	
	NRMS reproduction	0.6008	0.3200	0.3027	0.3645	+0.0048 [−0.0093, +0.0187] n.s.
	NRMS + category	0.6145	0.3240	0.2996	0.3675	+0.0078 [−0.0061, +0.0206] n.s.

Table 2: Test split, 2,000 impressions. Δ is the paired-bootstrap difference from retrieval only (1,000 resamples over impressions); "n.s." marks an interval that includes zero.

Ablation. Because the two NRMS models differ by exactly one component, their difference isolates it.

Dataset	nDCG@10: category-aware − reproduction	95% CI	Excludes zero?
ebnerd_small	+0.0161	[+0.0065, +0.0262]	yes
mind_small	+0.0030	[−0.0054, +0.0113]	no

The gain is claimed on EB-NeRD only; on MIND the interval includes zero.

4. Extended evaluation of the full pipeline (Q5)

All seven metrics with paired-bootstrap 95% intervals, computed by Assignment 1's harness unchanged, for retrieval-only and for retrieval plus the category-aware NRMS re-ranker.

	ebnerd_small		mind_small	
Metric	retrieval only	+ NRMS category	retrieval only	+ NRMS category
AUC	0.5505 [0.5357, 0.5646]	0.5895 [0.5751, 0.6013]	0.5985 [0.5849, 0.6113]	0.6145 [0.6023, 0.6271]
MRR	0.3475 [0.3344, 0.3604]	0.3728 [0.3601, 0.3855]	0.3187 [0.3050, 0.3332]	0.3240 [0.3103, 0.3379]
nDCG@5	0.3867 [0.3715, 0.4009]	0.4193 [0.4041, 0.4331]	0.3039 [0.2883, 0.3194]	0.2996 [0.2845, 0.3139]
nDCG@10	0.4676 [0.4554, 0.4800]	0.4897 [0.4776, 0.5016]	0.3597 [0.3455, 0.3743]	0.3675 [0.3540, 0.3816]
Diversity	0.1739 [0.1733, 0.1745]	0.1782 [0.1777, 0.1787]	0.2238 [0.2230, 0.2247]	0.2376 [0.2371, 0.2381]
Novelty	17.2840 [17.2503, 17.3167]	17.3224 [17.2882, 17.3546]	17.2391 [17.2038, 17.2730]	17.6126 [17.5900, 17.6339]
Coverage	0.0749 [0.0650, 0.0680]	0.0745 [0.0644, 0.0675]	0.0142 [0.0115, 0.0124]	0.0146 [0.0116, 0.0125]

Table 3: Coverage is the size of a union rather than an average, so its point estimate lies outside its bootstrap interval; the interval describes sampling variability only.

Slices. Cold-start vs warm (whether the user has any prior click) and head vs tail (an impression is "head" if a clicked article is among the popular ones, popularity measured on the test period; an analysis label, never a model input). nDCG@10 with 95% intervals:

Dataset	Slice	n	Retrieval only	+ NRMS category
ebnerd_small	cold start	0	no impressions	
	warm	2,000	0.4676 [0.4554, 0.4800]	0.4897 [0.4776, 0.5016]
	head	1,019	0.4525 [0.4360, 0.4693]	0.4768 [0.4589, 0.4951]
	tail	981	0.4833 [0.4658, 0.4990]	0.5032 [0.4863, 0.5202]
mind_small	cold start	62	0.3201 [0.2344, 0.4137]	0.3908 [0.3183, 0.4720]
	warm	1,938	0.3610 [0.3461, 0.3758]	0.3668 [0.3532, 0.3814]
	head	1,194	0.4180 [0.3995, 0.4366]	0.4537 [0.4355, 0.4709]
	tail	806	0.2734 [0.2499, 0.2973]	0.2398 [0.2196, 0.2601]

Observation. Cold-start gains most on MIND, but $n = 62$ is too small for the interval to be trusted; EB-NeRD has no cold-start users in its sample.

5. Serving and scale (Q4)

Measured at batch size one on CPU, 50 warm-up requests discarded and 1,000 measured. One request: pool a user vector, ANN top-200 over the corpus, BM25 over those candidates, assemble the feature row, score, tie-break.

	ebnerd_small	mind_small
p50 / p95 / **p99** latency (ms)	4.13 / 5.36 / **6.15**	5.61 / 6.26 / **6.68**
ANN search p99 (ms)	2.64	3.87
feature build p99 (ms)	1.41	1.50
re-ranker forward pass p99 (ms)	0.06	0.07
ANN index on disk (MB)	37.5	118.0
Embedding matrix in RAM (MB)	31.9	100.2
Feature store on disk (MB)	111.8	95.1
Feature store in RAM (MB)	724.6	366.8
Core-seconds per 1,000 queries	4.26	5.53
Sustainable QPS on 4 cores (p99 < 100 ms)	940	723

Table 4: One benchmark run on this machine; latencies vary between runs. Cost is given as core-seconds because we have no dated, citable instance price, and an invented price would be a number that traces to nothing.

The p99 target of 100 ms is met on both datasets; ANN search is the slowest stage at p99.

6. Where it breaks at 10×

1. **In-RAM inverted index and feature store** grow linearly with the corpus, so memory breaks first.
2. **Embedding matrix** (about 100 MB on MIND, about 1 GB at 10×) competes with the encoder for 4 GB of VRAM; scoring against a memory-mapped matrix is the fix.
3. **Per-candidate Python work** in the feature build scales with candidate count; a per-impression loop already broke before the model did in Assignment 1 (37 hours vs 14 minutes once vectorised).
4. **Train/serve skew:** `popularity_train` is non-zero for only 5.6% of MIND and 8.2% of EB-NeRD submission candidates (samples of 20,000 MIND lines and 50,000 EB-NeRD rows), and the EB-NeRD test file has no clicks, so `session_click_count` has no source. The NRMS ranker uses only embeddings and a category label, so it is the model we submit.

7. Anti-gaming (Q9)

Metrics with and without features unavailable at serving time. We add the dataset-wide aggregates `total_inviews`, `total_pageviews`, `total_read_time` to the MLP re-ranker in a labelled

ablation (EB-NeRD; MIND ships none of them).

EB-NeRD, MLP re-ranker	AUC	MRR	nDCG@5	nDCG@10
Serving-available features only (headline)	0.5369	0.3277	0.3675	0.4533
+ `total_*` aggregates (ablation)	0.5918	0.3679	0.4147	0.4897

Table 5: Adding the unavailable features changes nDCG@10 by +0.0364 [+0.0256, +0.0469] (95% CI).

No future-click leakage. Tests check per-feature values, per-row split membership and the neural model's history tensor; canary tests (including a click at exactly the impression time) show they can fail.

8. Codabench leaderboards (Q5, Q7)

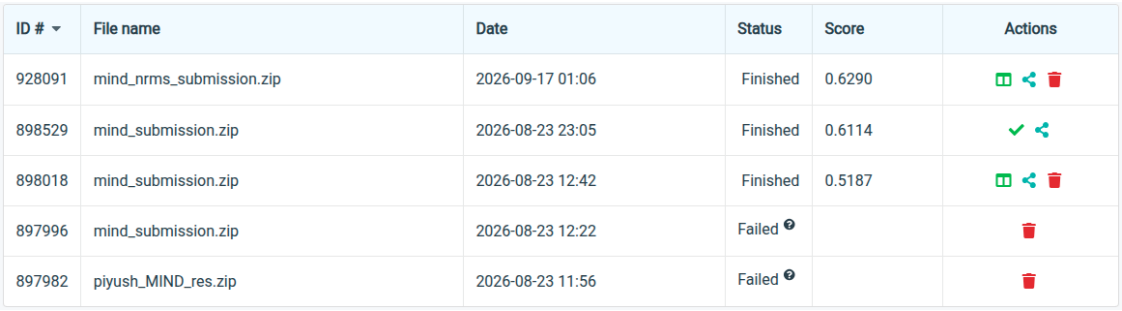
Both submissions use the category-aware NRMS ranker over the complete large test sets (2,370,727 MIND impressions; 13,536,710 EB-NeRD rows).

Competition	Submission	AUC	MRR	nDCG@5	nDCG@10
MIND	A2: NRMS + category	0.6290	0.2905	0.3122	0.3700
	A1: BM25 + semantic hybrid	0.6114	0.2969	0.3179	0.3732
EB-NeRD	A2: NRMS + category	first upload failed; resubmission running, no score yet			
	A1: semantic retrieval	0.5508	0.3510	0.3916	0.4679

Table 6: Scores returned by the Codabench scorers. Grading is not on rank; Assignment 1 rows are shown for reference.

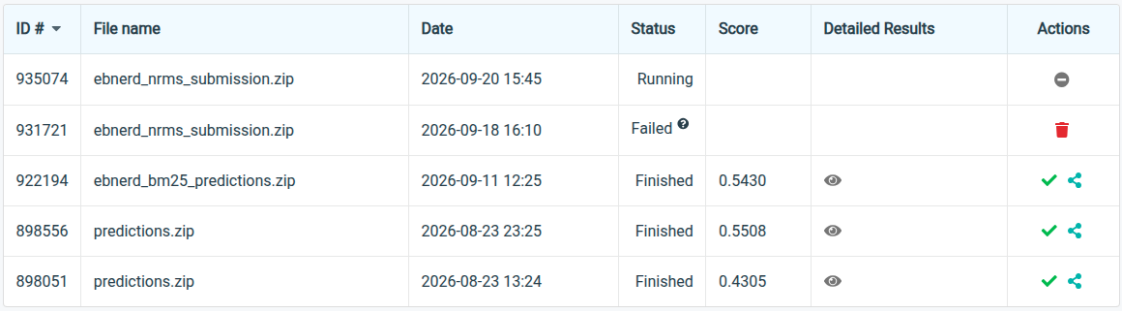
MIND. NRMS is ahead of the Assignment 1 hybrid on AUC and slightly behind on MRR, nDCG@5 and nDCG@10.

EB-NeRD. The first NRMS submission (#931721) failed at Codabench and has no score (all status fields null). We verified the file: all 13,536,710 rows have the right number of ranks, in order, as a valid permutation, in a single `predictions.txt` that passes `unzip -t`. The same file was resubmitted (#935074) and had not finished when this report was written, so the NRMS result for EB-NeRD is reported offline only (Sections 3 and 4).



ID #	File name	Date	Status	Score	Actions
928091	mind_nrms_submission.zip	2026-09-17 01:06	Finished	0.6290	
898529	mind_submission.zip	2026-08-23 23:05	Finished	0.6114	
898018	mind_submission.zip	2026-08-23 12:42	Finished	0.5187	
897996	mind_submission.zip	2026-08-23 12:22	Failed		
897982	piyush_MIND_res.zip	2026-08-23 11:56	Failed		

Figure 1: MIND competition, "My submissions": the NRMS submission (top row) scored 0.6290.



ID #	File name	Date	Status	Score	Detailed Results	Actions
935074	ebnerd_nrms_submission.zip	2026-09-20 15:45	Running			
931721	ebnerd_nrms_submission.zip	2026-09-18 16:10	Failed			
922194	ebnerd_bm25_predictions.zip	2026-09-11 12:25	Finished	0.5430		
898556	predictions.zip	2026-08-23 23:25	Finished	0.5508		
898051	predictions.zip	2026-08-23 13:24	Finished	0.4305		

Figure 2: EB-NeRD competition, "My submissions": the first NRMS submission (#931721) shows *Failed*; the resubmission (#935074, top row) was still *Running* when this report was written.

Reproducibility

```
make setup && make test              # no dataset and no GPU required
make clean-derived && make all       # every Assignment 1 number
make a2-all                          # the A2 stages (these train, so GPU-only)
make submit-nrms                     # the two Codabench prediction files
```